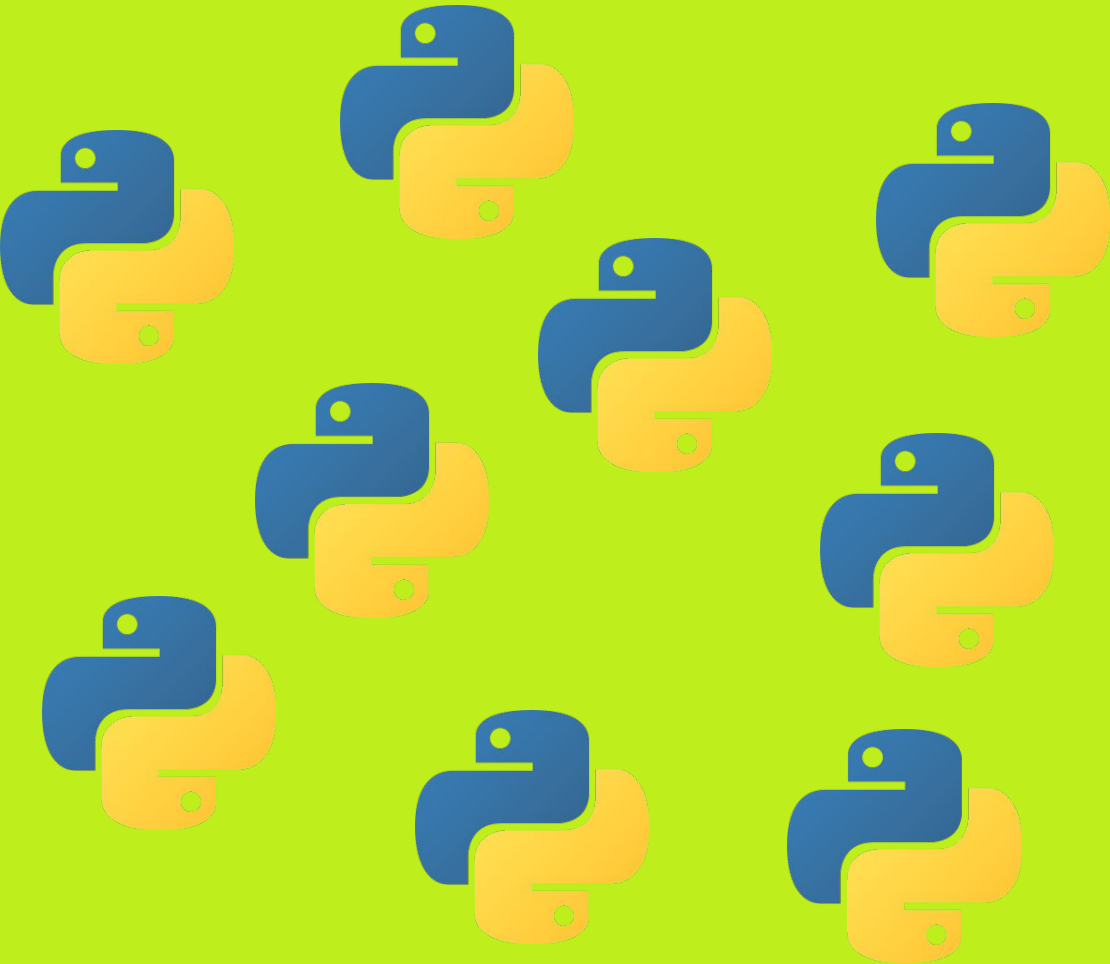


PYTHON: A QUICK



**REVIEW OF A
PROGRAMMING
LANGUAGE**





DATA TYPES

VALUES AND TYPES

INTEGERS: WHOLE MATHEMATICAL VALUES (EX: 8, 23, -12)

FLOATS: DECIMAL POINT VALUES (EX: 8.6, 23.2, -12.0001)

STRINGS: TEXT VALUES (EX: “HELLO”, “WORLD”, “14”)

TYPE():

```
>>> type('Hello, World!')  
<class 'str'>  
>>> type(17)  
<class 'int'>
```

PRINT()

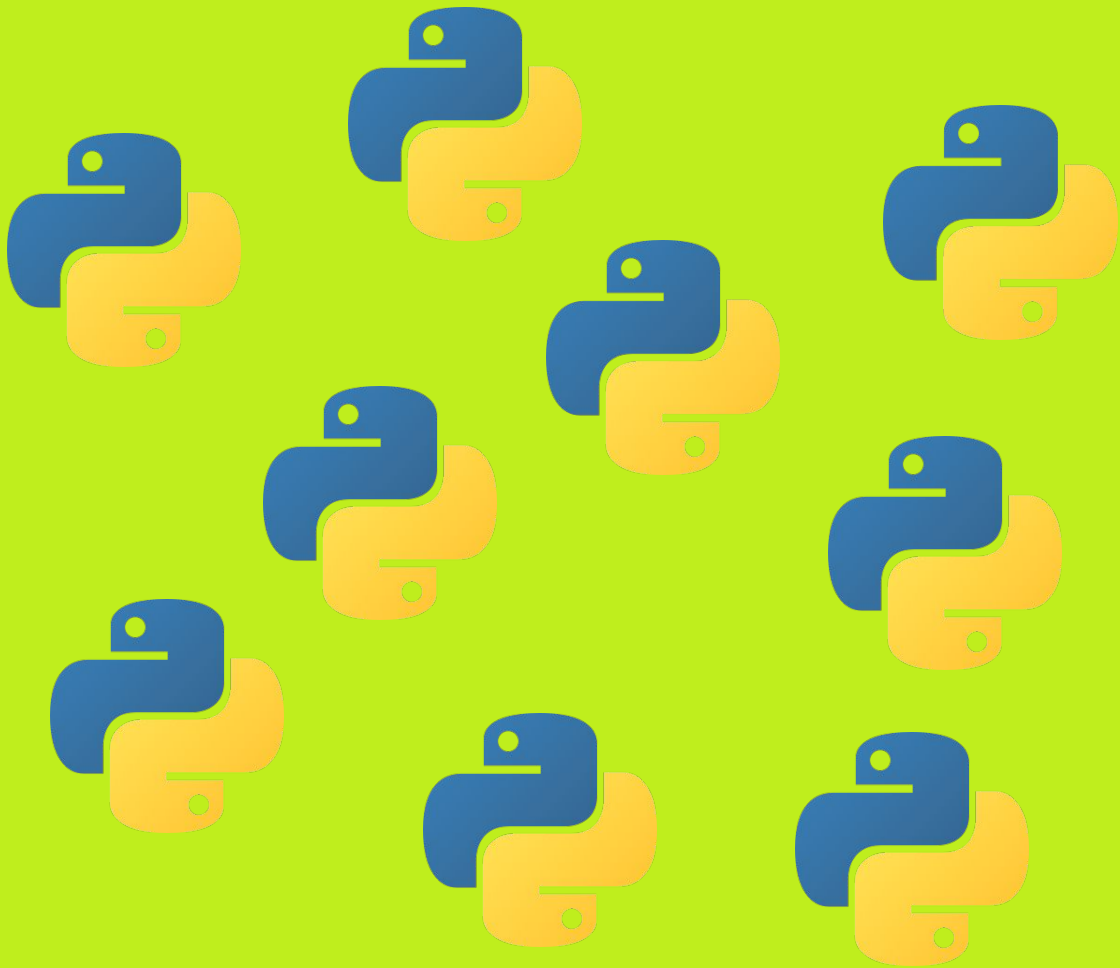
CONCATENATION:

```
a = "Hello"  
b = "World"  
c = a + " " + b  
print(c)  
# Output: Hello World
```

F(STRING):

```
x = 10  
print(f"The price is {x} dollars.")
```

VARIABLES



A VARIABLE IS A NAME THAT REFERS TO A VALUE.

ASSIGNMENT STATEMENT:

```
>>> message = 'And now for something completely different'
>>> n = 17
>>> pi = 3.1415926535897931
```

PRINT:

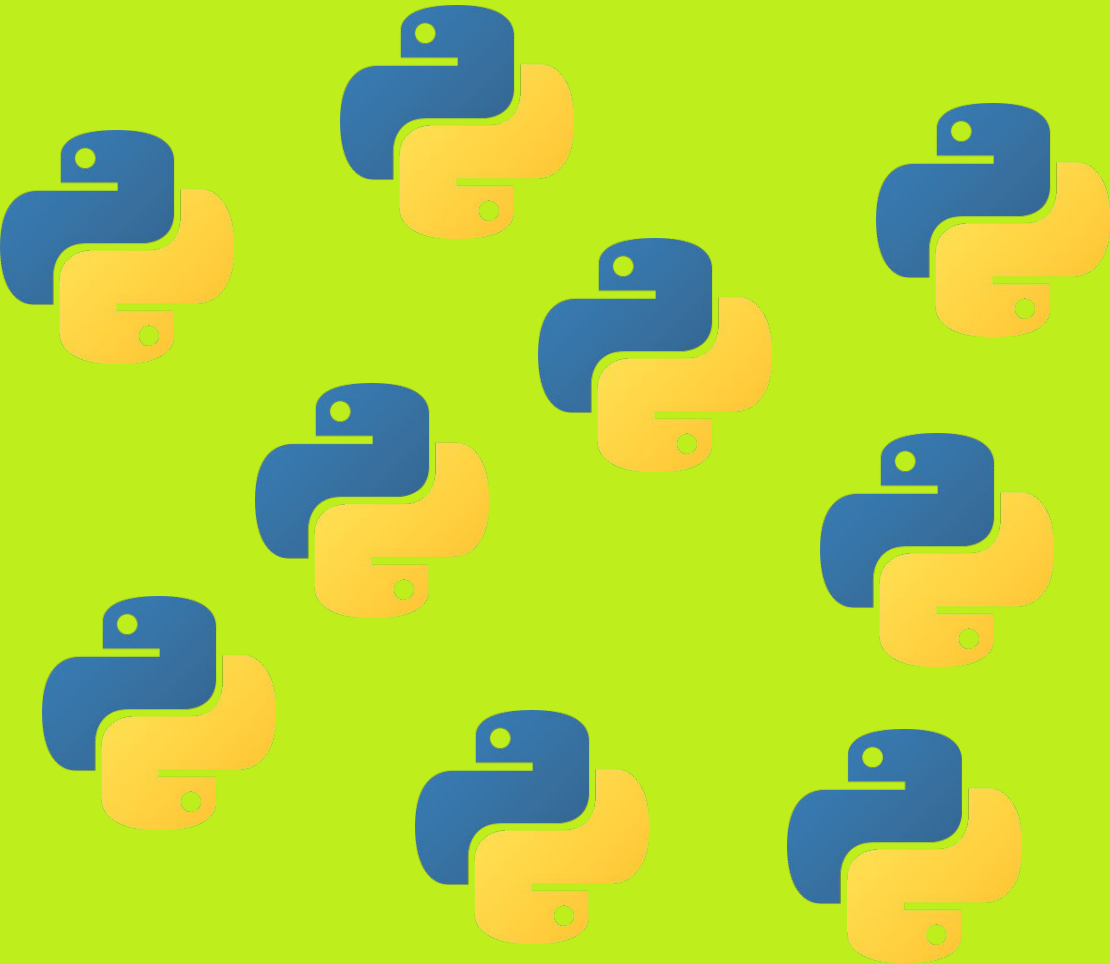
```
>>> print(n)
17
>>> print(pi)
3.141592653589793
```

TYPE:

```
>>> type(message)
<class 'str'>
>>> type(n)
<class 'int'>
>>> type(pi)
<class 'float'>
```

PYTHON KEYWORDS (CAN'T BE USED AS VARIABLE NAMES)

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield



CONDITIONALS

IF/ELIF

CONDITIONAL EXECUTION:

```
if x > y:  
    print(x)  
    print(y)
```

ALTERNATIVE EXECUTION:

```
if x % 2 == 0:  
    print('x is even')  
else:  
    print('x is odd')
```

CHAINED CONDITIONAL:

```
if x < y:  
    print('x is less than y')  
elif x > y:  
    print('x is greater than y')  
else:  
    print('x and y are equal')
```

NESTED CONDITIONAL:

```
if x == y:  
    print('x and y are equal')  
else:  
    if x < y:  
        print('x is less than y')  
    else:  
        print('x is greater than y')
```

OPERATORS

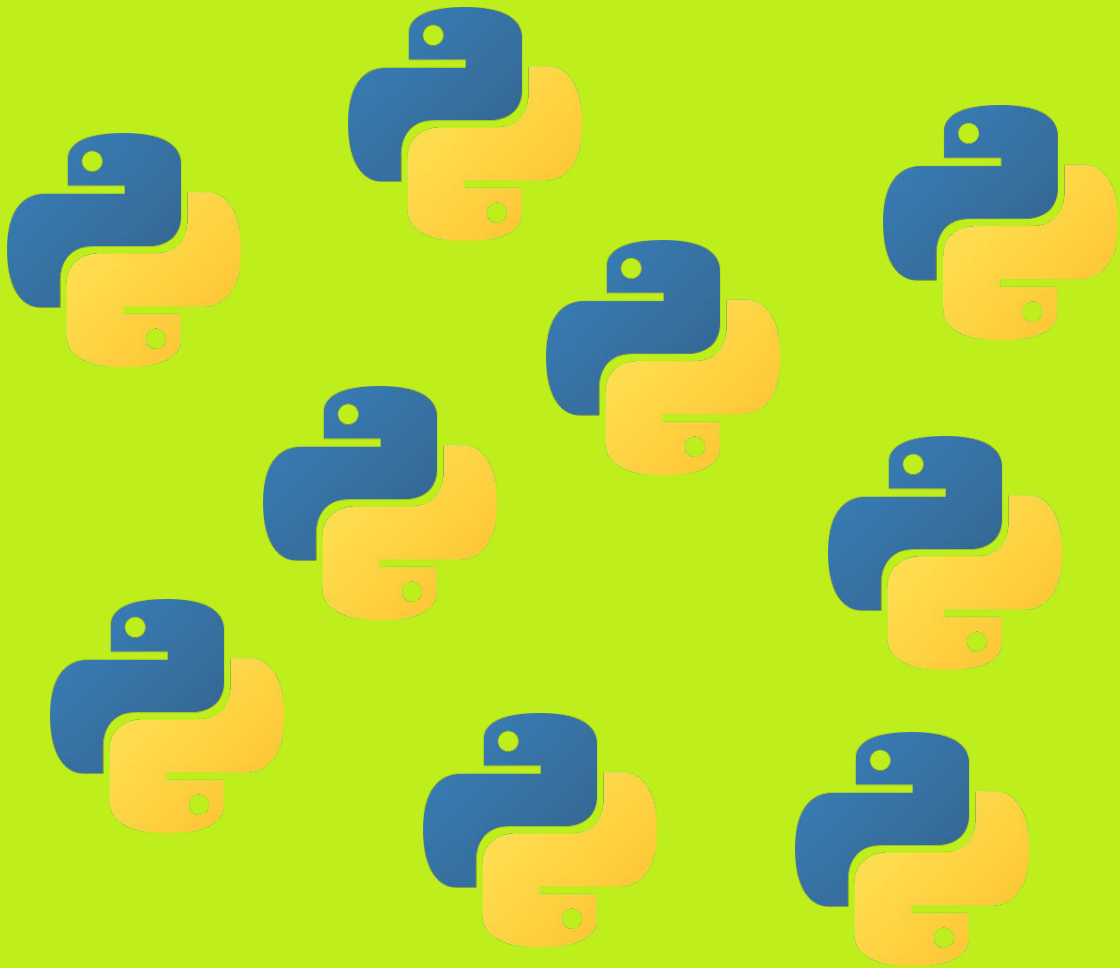
COMPARISON OPERATORS:

<code>x != y</code>	# x is not equal to y
<code>x > y</code>	# x is greater than y
<code>x < y</code>	# x is less than y
<code>x >= y</code>	# x is greater than or equal to y
<code>x <= y</code>	# x is less than or equal to y
<code>x is y</code>	# x is the same as y
<code>x is not y</code>	# x is not the same as y

LOGICAL OPERATORS:

and	Returns True if both operands are true.	<code>x and y</code>	<code>(5 > 2) and (3 < 5)</code>	True
or	Returns True if at least one operand is true.	<code>x or y</code>	<code>(5 > 2) or (3 > 5)</code>	True
not	Reverses the logical state of its operand.	<code>not x</code>	<code>not(5 > 2)</code>	False

FUNCTIONS



USER-DEFINED FUNCTIONS

DEFINING :

```
>>> def print_lyrics():  
...     print("I'm a lumberjack, and I'm okay.")  
...     print('I sleep all night and I work all day.')  
... 
```

CALLING :

```
>>> print_lyrics()  
I'm a lumberjack, and I'm okay.  
I sleep all night and I work all day.
```

NESTING :

```
def repeat_lyrics():  
    print_lyrics()  
    print_lyrics()
```

```
>>> repeat_lyrics()  
I'm a lumberjack, and I'm okay.  
I sleep all night and I work all day.  
I'm a lumberjack, and I'm okay.  
I sleep all night and I work all day.
```

PARAMETERS

```
def print_twice(bruce):  
    print(bruce)  
    print(bruce)
```

```
>>> print_twice('Spam')  
Spam  
Spam  
>>> print_twice(17)  
17  
17  
>>> import math  
>>> print_twice(math.pi)  
3.141592653589793  
3.141592653589793
```

BUILT-IN FUNCTIONS

Built-in Functions

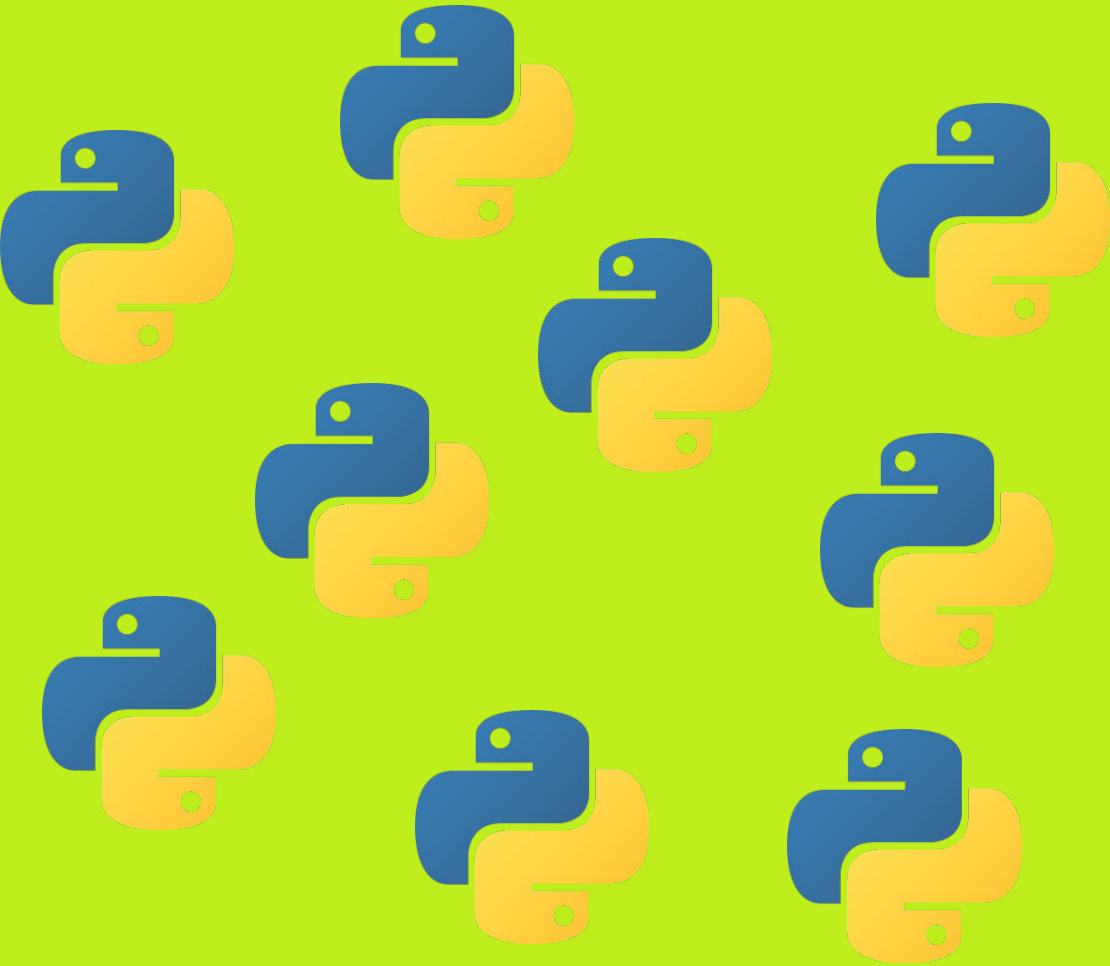
A <u>abs()</u> <u>aiter()</u> <u>all()</u> <u>anext()</u> <u>any()</u> <u>ascii()</u>	E <u>enumerate()</u> <u>eval()</u> <u>exec()</u>	L <u>len()</u> <u>list()</u> <u>locals()</u>	R <u>range()</u> <u>repr()</u> <u>reversed()</u> <u>round()</u>
B <u>bin()</u> <u>bool()</u> <u>breakpoint()</u> <u>bytearray()</u> <u>bytes()</u>	F <u>filter()</u> <u>float()</u> <u>format()</u> <u>frozenset()</u>	M <u>map()</u> <u>max()</u> <u>memoryview()</u> <u>min()</u>	S <u>set()</u> <u>setattr()</u> <u>slice()</u> <u>sorted()</u> <u>staticmethod()</u> <u>str()</u> <u>sum()</u> <u>super()</u>
C <u>callable()</u> <u>chr()</u> <u>classmethod()</u> <u>compile()</u> <u>complex()</u>	H <u>hasattr()</u> <u>hash()</u> <u>help()</u> <u>hex()</u>	N <u>next()</u>	T <u>tuple()</u> <u>type()</u>
D <u>delattr()</u> <u>dict()</u> <u>dir()</u> <u>divmod()</u>	I <u>id()</u> <u>input()</u> <u>int()</u> <u>isinstance()</u> <u>issubclass()</u> <u>iter()</u>	O <u>object()</u> <u>oct()</u> <u>open()</u> <u>ord()</u>	V <u>vars()</u>
		P <u>pow()</u> <u>print()</u> <u>property()</u>	Z <u>zip()</u>
			<u>__import__()</u>

<https://docs.python.org/3/library/functions.html>

RETURN

The `return` statement in Python is used inside a function to send a value or values back to the part of the program that called the function and to immediately exit the function.

```
def add_numbers(a, b):  
    sum_result = a + b  
    return sum_result # Returns the value of sum_result  
  
# Call the function and store the returned value in a variable  
total = add_numbers(5, 3)  
  
# Print the stored value  
print(total)  
# Output: 8  
  
# Example of a function exiting early based on a condition  
def validate_age(age):  
    if age < 0:  
        return False # Exits the function immediately  
    return True # Reached if the condition is not met
```



ITERATION

WHILE LOOPS

```
n = 5
while n > 0:
    print(n)
    n = n - 1
print('Blastoff!')
```

```
while True:
    line = input('> ')
    if line == 'done':
        break
    print(line)
print('Done!')
```

Code: <https://www.py4e.com/code3/copytildone1.py>

SOMETIMES WE WANT TO LOOP THROUGH A SET OF THINGS SUCH AS A LIST OF WORDS, THE LINES IN A FILE, OR A LIST OF NUMBERS.

WHEN WE HAVE A LIST OF THINGS TO LOOP THROUGH, WE CAN CONSTRUCT A DEFINITE LOOP USING A FOR STATEMENT.

WE CALL THE WHILE STATEMENT AN INDEFINITE LOOP BECAUSE IT SIMPLY LOOPS UNTIL SOME CONDITION BECOMES FALSE, WHEREAS THE FOR LOOP IS LOOPING THROUGH A KNOWN SET OF ITEMS SO IT RUNS THROUGH AS MANY ITERATIONS AS THERE ARE ITEMS IN THE SET.

FOR LOOPS

```
friends = ['Joseph', 'Glenn', 'Sally']  
for friend in friends:  
    print('Happy New Year:', friend)  
print('Done!')
```

In Python terms, the variable `friends` is a list of three strings and the `for` loop goes through the list and executes the body once for each of the three strings in the list resulting in this output:

In particular, `friend` is the *iteration variable* for the `for` loop. The variable `friend` changes for each iteration of the loop and controls when the `for` loop completes. The *iteration variable* steps successively through the three strings stored in the `friends` variable.

```
Happy New Year: Joseph  
Happy New Year: Glenn  
Happy New Year: Sally  
Done!
```

NESTED LOOPS

A common example: generating all combinations of items from two lists

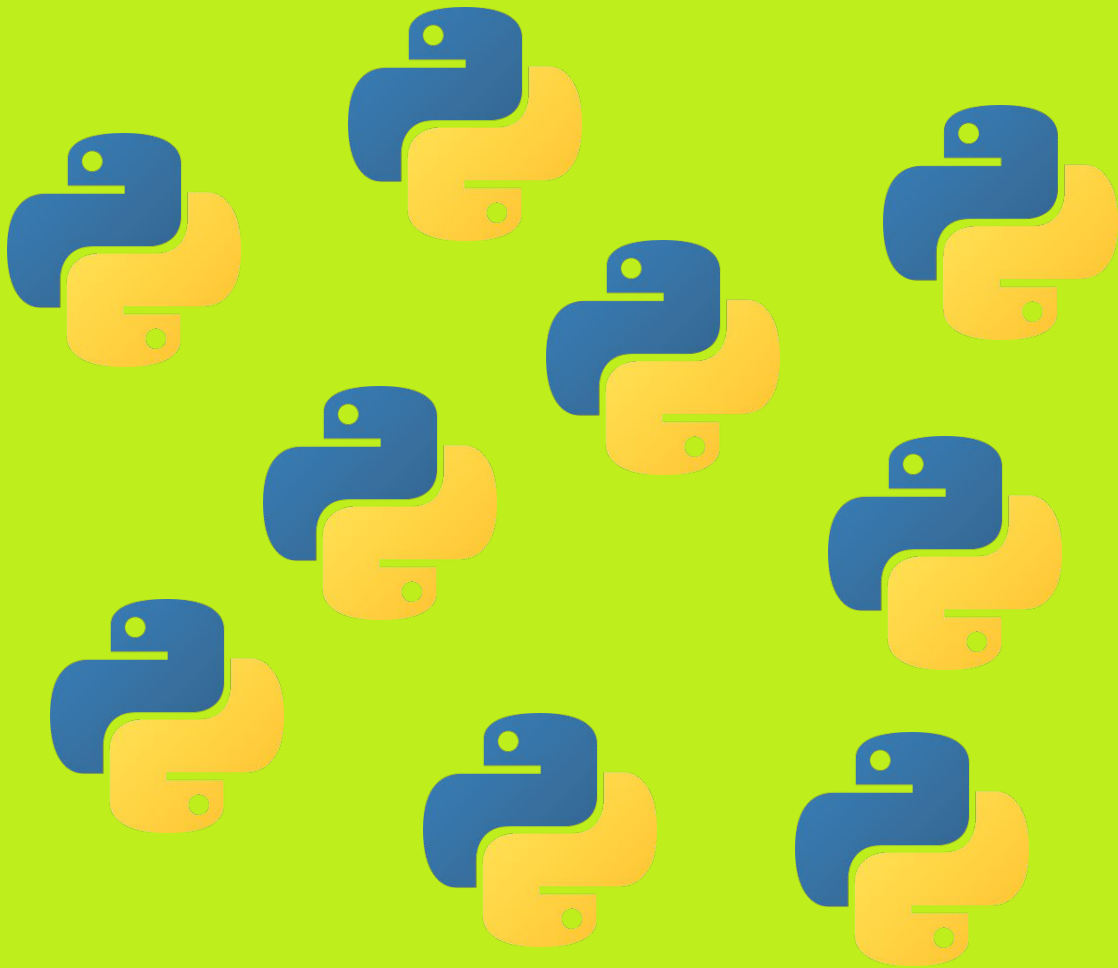
`adjectives = ["red", "big", "tasty"]`

`fruits = ["apple", "banana", "cherry"]`

```
for adj in adjectives:    # Outer loop iterates through adjectives
    for fruit in fruits:  # Inner loop iterates through fruits for each adjective
        print(adj, fruit)
```

```
red apple
red banana
red cherry
big apple
big banana
big cherry
tasty apple
tasty banana
tasty cherry
```

LISTS



A LIST IS A MUTABLE INDEX-BASED SEQUENCE

```
>>> cheeses = ['Cheddar', 'Edam', 'Gouda']  
>>> numbers = [17, 123]  
>>> empty = []  
>>> print(cheeses, numbers, empty)  
['Cheddar', 'Edam', 'Gouda'] [17, 123] []
```

Lists: Index-Based

Definition: An ordered sequence of elements, accessed by their numerical position (starting at 0).

Usage: Used when the order of elements matters.

Access: `my_list[0]` fetches the first item.

Time Complexity: $O(1)$ for accessing an element by its index.

SLICING A LIST

The slice operator also works on lists:

```
>>> t = ['a', 'b', 'c', 'd', 'e', 'f']
>>> t[1:3]
['b', 'c']
>>> t[:4]
['a', 'b', 'c', 'd']
>>> t[3:]
['d', 'e', 'f']
```

If you omit the first index, the slice starts at the beginning. If you omit the second, the slice goes to the end. So if you omit both, the slice is a copy of the whole list.

```
>>> t[:]
['a', 'b', 'c', 'd', 'e', 'f']
```

Since lists are mutable, it is often useful to make a copy before performing operations that fold, spindle, or mutilate lists.

A slice operator on the left side of an assignment can update multiple elements:

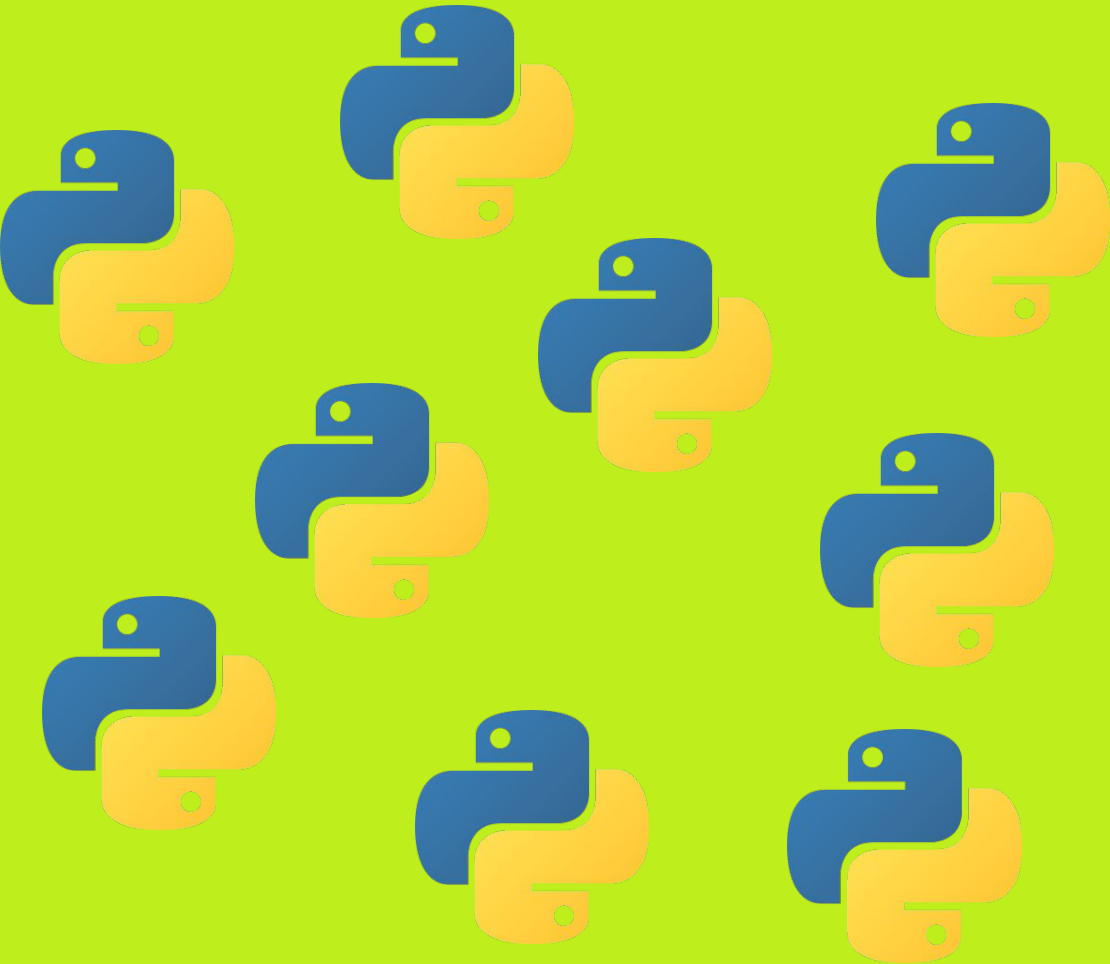
```
>>> t = ['a', 'b', 'c', 'd', 'e', 'f']
>>> t[1:3] = ['x', 'y']
>>> print(t)
['a', 'x', 'y', 'd', 'e', 'f']
```

LIST METHODS

Python List Methods

Input	Method	Output
	<code>.append()</code>	
	<code>.insert(1, )</code>	
	<code>.pop(1)</code>	
	<code>.remove()</code>	
	<code>.reverse()</code>	
	<code>.sort()</code>	
	<code>.index()</code>	2
	<code>.count()</code>	2

Method	Application
<code>append()</code>	Add one or more items at the end of list
<code>insert()</code>	Insert an item at the defined index
<code>extend()</code>	Add all items of a list to the end of this list
<code>remove()</code>	Removes an item with given value from the list
<code>pop()</code>	Removes an item with given index from the list
<code>clear()</code>	Remove all items from the list
<code>index()</code>	Returns index of the item in list with given value
<code>copy()</code>	Returns a copy of the list to a target reference
<code>count()</code>	Returns number of items with specified value
<code>sort()</code>	Sort a list by default in ascending order; Can also sort a list in reverse order or sort by a callable key
<code>reverse()</code>	Reverses the order of a list



DICTIONARIES

A DICTIONARY IS A MUTABLE KEY-BASED SEQUENCE

```
>>> eng2sp = {'one': 'uno', 'two': 'dos', 'three': 'tres'}  
>>> print(eng2sp)  
{'one': 'uno', 'two': 'dos', 'three': 'tres'}
```

Dictionaries: Key-Based

Definition: An unordered collection (as of Python 3.6 and earlier, ordered in 3.7+) of key-value pairs, accessed by a unique key rather than a number.

Usage: Used for fast lookups, mappings, and structured data.

Access: my_dict['name'] fetches the value associated with the key 'name'.

Time Complexity: $O(1)$ for looking up a value by its key (on average).

DICTIONARY METHODS

Dictionary in Python PYNATIVE.COM

Unordered collections of unique values stored in (Key-Value) pairs.

```
d = {'a': 10, 'b': 20, 'c': 30}
```



d['a']



d['b']



d['c']

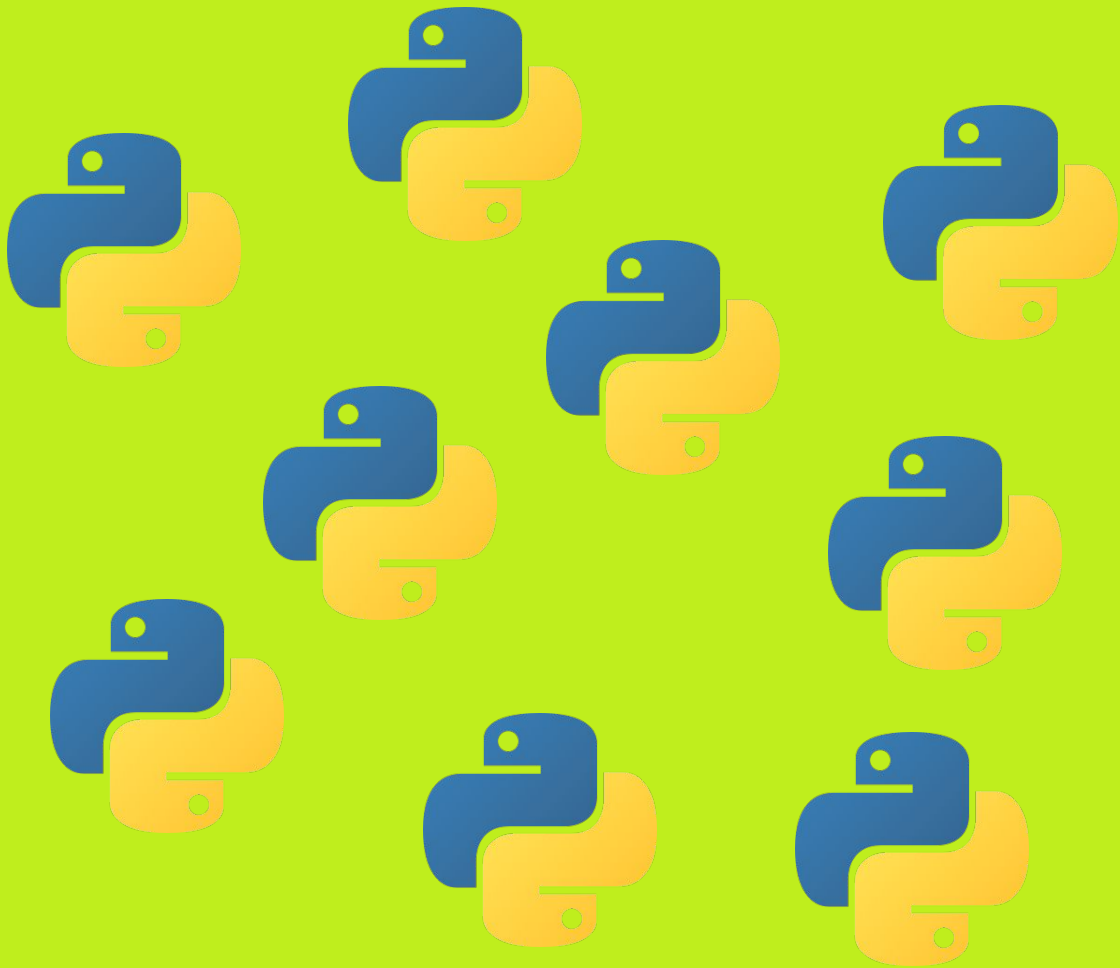
- ✓ **Unordered:** The items in dict are stored without any index value
- ✓ **Unique:** Keys in dictionaries should be Unique
- ✓ **Mutable:** We can add/Modify/Remove key-value after the creation

Dictionary methods

The dict type is a mutable mapping of hashable keys to values.

clear()	Remove all items from the dictionary
copy()	Return a shallow copy of the dictionary.
fromkeys()	Returns a new dictionary with keys from a sequence and values set to a specified value.
get()	Returns the value for a specified key. If the key is not found, it returns None (or a default value if specified).
items()	Returns a view object that displays a list of a dictionary's key-value tuple pairs.
keys()	Returns a view object that displays a list of all the dictionary's keys.
pop()	Removes and returns the value for the specified key.
popitem()	Removes and returns a random (key, value) pair from the dictionary. (Since Python 3.7, the last item is removed in insertion order.)
setdefault()	If the key exists, it returns the value. If the key doesn't exist, it inserts the key with a specified default value.
update()	Updates the dictionary with elements from another dictionary or from an iterable of key-value pairs.
values()	Returns a view object that displays a list of all the values in the dictionary.

TUPLES



A TUPLE IS AN IMMUTABLE INDEX-BASED SEQUENCE

```
>>> t = ('a', 'b', 'c', 'd', 'e')
```

Key Characteristics of Tuples:

Immutable: Once created, you cannot add, remove, or change elements.

Ordered: The elements have a defined, unchanging order.

Indexed: Elements are accessed by their position, starting at index 0.

Syntax: Typically enclosed in parentheses, e.g., `my_tuple = (1, 2, 3)`.

Heterogeneous: Can contain different data types.

TUPLES VS OTHER DATA COLLECTIONS

Common Uses:

Data Integrity: Used when data should not be changed throughout the program's execution (e.g., configurations).

Fixed Records: Storing related, but different, pieces of data together, like (x, y) coordinates.

Return Values: Returning multiple values from a single function.

Dictionary Keys: Because they are hashable, tuples can be used as keys in dictionaries, whereas lists cannot.

Tuple vs. List:

List: Mutable [1, 2, 3], slower performance.

Tuple: Immutable (1, 2, 3), faster performance.

Mathematical/Database Definition: In mathematics, an (n) -tuple is a sequence of (n) elements. In databases, a tuple represents a single row (or record) in a table.





REFERENCES:

<https://www.py4e.com/lessons>